Notas de Clase: ResNet

Aprendizaje Residual Profundo para Reconocimiento de Imágenes

Aldo Camargo, Ph.D.

July 15, 2026

Abstract

Estas notas cubren ResNet (He, Zhang, Ren y Sun, 2016), ganadora de ILSVRC-2015, que introdujo las *conexiones residuales* (*skip connections*) y permitió entrenar de forma confiable redes de cientos — incluso miles — de capas. Derivamos el *problema de degradación* que motiva el diseño, la formulación matemática del aprendizaje residual, los bloques básico y *bottleneck*, las variantes de la conexión de atajo (*shortcut*) para cambios de dimensión, la arquitectura completa de ResNet-18/34/50/101/152, y cerramos con la interpretación de ResNet como un ensamble implícito de redes de profundidad variable y su legado en arquitecturas posteriores.

Contents

1 El Problema de Degradación **1**

2 Aprendizaje Residual **2**
2.1 La Idea Central 2
2.2 Bloque Básico vs. Bloque Bottleneck 3
2.3 Conexiones de Atajo con Cambio de Dimensión 4

3 Arquitectura Completa **4**

4 Configuración de Entrenamiento **5**

5 Resultados **5**

6 ResNet como Ensamble Implícito **5**

7 Legado y Trayectoria de Diseño **6**

8 Ejercicios **6**

9 Referencias **7**

1 El Problema de Degradación

Las notas sobre VGGNet ya insinuaban un límite: pasar de 16 a 19 capas apenas mejoró el desempeño. He et al. investigaron sistemáticamente qué ocurre al seguir apilando capas convolucionales simples (*planas*, sin atajos) y encontraron un fenómeno contraintuitivo.

Definition: Problema de Degradación

Al aumentar la profundidad de una red *plana* (sin conexiones residuales) más allá de cierto punto, tanto el error de **entrenamiento** como el de prueba **aumentan** — no solo el de prueba, como predeciría el sobreajuste clásico. Por ejemplo, una red plana de 56 capas obtiene mayor error de entrenamiento que una de 20 capas sobre CIFAR-10, a pesar de que la red de 56 capas tiene estrictamente mayor capacidad representacional.

Intuition: ¿Por qué es esto sorprendente?

Considere una red profunda $\mathcal{N}_{\text{profunda}}$ construida a partir de una red superficial $\mathcal{N}_{\text{superficial}}$ agregando capas adicionales que implementan la función identidad. En teoría, $\mathcal{N}_{\text{profunda}}$ debería poder alcanzar *al menos* el mismo error de entrenamiento que $\mathcal{N}_{\text{superficial}}$, simplemente aprendiendo la identidad en las capas extra. Empíricamente esto no ocurre: los optimizadores basados en descenso de gradiente (SGD) tienen dificultad para aprender mapeos de identidad a través de pilas de capas no lineales. El problema de degradación es, por tanto, un problema de **optimización**, no de capacidad representacional ni de sobreajuste.

2 Aprendizaje Residual

2.1 La Idea Central

En lugar de esperar que un conjunto de capas apiladas aprenda directamente un mapeo deseado $\mathcal{H}(x)$, ResNet las reformula para que aprendan la función **residual** $\mathcal{F}(x) = \mathcal{H}(x) - x$, y luego reconstruyen la salida deseada sumando la entrada original:

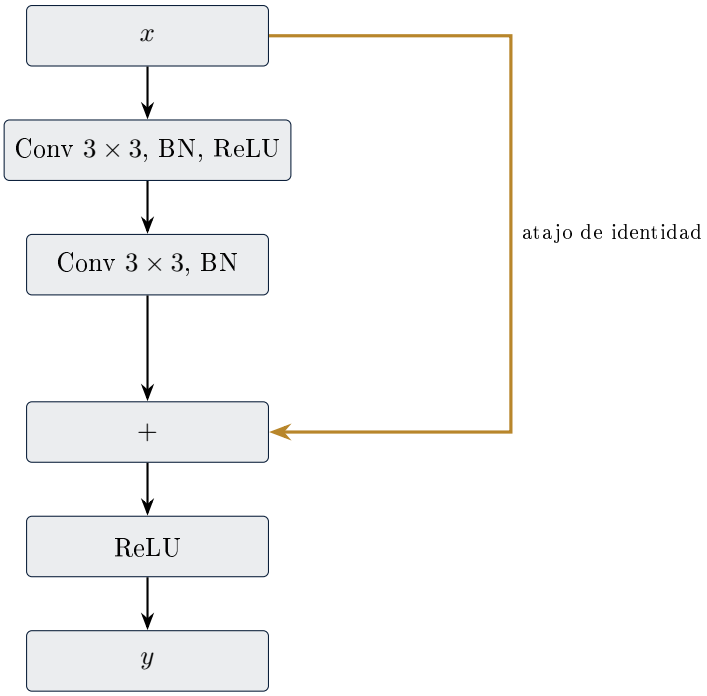
$$y = \mathcal{F}(x, \{W_i\}) + x.$$

Definition: Bloque Residual

Un bloque residual calcula $y = \mathcal{F}(x, \{W_i\}) + x$, donde $\mathcal{F}$ representa el mapeo residual a aprender (típicamente dos o tres capas convolucionales con normalización por lotes y ReLU), y la suma con x se implementa mediante una **conexión de atajo** (*shortcut connection*) que salta esas capas. Cuando las dimensiones de x y $\mathcal{F}(x)$ coinciden, el atajo es la función identidad pura — sin parámetros adicionales ni costo computacional extra.

Intuition: Por qué esta reformulación facilita la optimización

Si el mapeo óptimo para un bloque dado es cercano a la identidad, es mucho más fácil para el optimizador empujar los pesos de $\mathcal{F}$ hacia cero (de modo que $y \approx x$) que aprender una transformación no lineal completa que reproduzca la identidad desde cero. En otras palabras, ResNet no elimina el problema de aprender la identidad; lo convierte en un problema mucho más fácil (acercar pesos a cero) en lugar de uno difícil (ajustar una composición de no linealidades para que se cancelen exactamente).



2.2 Bloque Básico vs. Bloque Bottleneck

Definition: Bloque Básico (*Basic Block*)

Usado en ResNet-18 y ResNet-34: dos convoluciones 3×3 consecutivas (cada una seguida de BatchNorm), con la conexión de atajo sumada antes de la ReLU final. Mantiene el mismo número de canales a lo largo del bloque (excepto en los puntos de reducción espacial).

Definition: Bloque Bottleneck

Usado en ResNet-50/101/152 para controlar el costo computacional a mayor profundidad: una secuencia de tres convoluciones, $1 \times 1 \to 3 \times 3 \to 1 \times 1$. La primera convolución 1×1 **reduce** el número de canales (típicamente por un factor de 4), la convolución 3×3 opera sobre ese espacio de menor dimensión, y la última convolución 1×1 **restaura** el número de canales original antes de sumar el atajo. Esta es exactamente la misma estrategia de reducción de dimensionalidad que motivó los módulos Inception de GoogLeNet, aplicada aquí para permitir apilar muchos más bloques sin que el costo computacional se dispare.

Worked Example: Costo computacional: bloque básico vs. bottleneck a 256 canales

Para un bloque básico con $C = 256$ canales de entrada/salida (dos convoluciones 3×3):

$$\text{Parámetros} \approx 2 \times (3^2 \times 256 \times 256) = 1{,}179{,}648.$$

Para un bloque bottleneck equivalente (reducción a $64 = 256/4$ canales internos):

$$\underbrace{1 \times 1 \times 256 \times 64}_{\text{reducción}} + \underbrace{3 \times 3 \times 64 \times 64}_{\text{conv central}} + \underbrace{1 \times 1 \times 64 \times 256}_{\text{restauración}} = 16{,}384 + 36{,}864 + 16{,}384 = 69{,}632.$$

El bloque bottleneck logra una reducción de parámetros de aproximadamente 17× para el mismo número de canales de entrada/salida, lo que hace viable apilar decenas de bloques adicionales (como en ResNet-101 o ResNet-152) sin un costo prohibitivo.

2.3 Conexiones de Atajo con Cambio de Dimensión

Cuando el bloque residual cambia el número de canales o reduce la resolución espacial (típicamente al inicio de cada "etapa" de la red, con stride 2), la suma directa $\mathcal{F}(x) + x$ ya no es posible porque las dimensiones no coinciden. El artículo original propone y compara dos soluciones:

- **Opción A (atajo de identidad con relleno de ceros):** se aplica un submuestreo (p. ej., *strided pooling*) y se rellenan con ceros los canales adicionales. No agrega parámetros.
- **Opción B (atajo de proyección):** se aplica una convolución 1×1 con stride 2 sobre el atajo, aprendiendo la proyección necesaria. Agrega un número pequeño de parámetros pero generalmente da mejores resultados y es la opción estándar en las implementaciones modernas (incluida la de este curso).

3 Arquitectura Completa

Capa	**ResNet-18**	**ResNet-34**	**ResNet-50**	**ResNet-101**	**ResNet-152**
Conv inicial	Conv $7 \times 7/2$, 64 canales, MaxPool $3 \times 3/2$				
Etapa 1 (conv2_x)	[Básico]×2	[Básico]×3	[Bottleneck]×3	[Bottleneck]×3	[Bottleneck]×3
Etapa 2 (conv3_x)	[Básico]×2	[Básico]×4	[Bottleneck]×4	[Bottleneck]×4	[Bottleneck]×8
Etapa 3 (conv4_x)	[Básico]×2	[Básico]×6	[Bottleneck]×6	[Bottleneck]×23	[Bottleneck]×36
Etapa 4 (conv5_x)	[Básico]×2	[Básico]×3	[Bottleneck]×3	[Bottleneck]×3	[Bottleneck]×3
Clasificador	Global Average Pooling → FC 1000 → Softmax				
Profundidad total	18	34	50	101	152
Parámetros	11.7M	21.8M	25.6M	44.5M	60.2M

Nótese que cada etapa reduce la resolución espacial a la mitad (stride 2 en la primera convolución de la etapa) mientras **duplica** el número de canales — el mismo patrón de diseño "duplicar canales al reducir resolución" que vimos en VGGNet.

Definition: Regla de Diseño de Canales/Resolución

Para dos capas con el mismo tamaño de mapa de características, el número de filtros es el mismo. Si el tamaño del mapa se reduce a la mitad, el número de filtros se duplica, de modo que la complejidad temporal por capa se mantenga aproximadamente constante — un principio de diseño heredado de VGGNet y GoogLeNet.

4 Configuración de Entrenamiento

Hiperparámetro	Valor
Optimizador	SGD con momentum 0.9
Weight decay	1×10^{-4}
Tamaño de lote	256
Tasa de aprendizaje	0.1, dividida entre 10 al estancarse el error (varias veces)
Normalización	Batch Normalization después de cada convolución, antes de la ReLU (Section 5, notas de AlexNet: reemplaza por completo a LRN)
Inicialización	Inicialización de He (*Kaiming*), diseñada específicamente para capas con activaciones ReLU
Épocas	Hasta 60×10^4 iteraciones

5 Resultados

Modelo	Capas	Parámetros	Error top-5 (ILSVRC-2015)
VGG-16	16	138M	8.8%
GoogLeNet	22	6.8M	7.9%
ResNet-34 (plano, sin atajos)	34	21.8M	mayor error que ResNet-18 (degradación)
ResNet-34 (con atajos)	34	21.8M	$\approx 5.7\%$ (single model, val)
ResNet-152 (ensamble)	152	60.2M	3.57% (ganador ILSVRC-2015)

El resultado empírico clave

La comparación más importante del artículo no es ResNet contra VGGNet o GoogLeNet, sino **ResNet-34 plano contra ResNet-34 con conexiones residuales**, ambos con idéntica arquitectura salvo por los atajos. La versión plana sufre del problema de degradación (peor que su contraparte de 18 capas); la versión residual no solo evita la degradación, sino que supera claramente a su contraparte de 18 capas — demostrando de forma controlada que las conexiones residuales son la causa directa de la mejora, no simplemente el aumento de capacidad.

6 ResNet como Ensamble Implícito

Una interpretación influyente (Veit et al., 2016) sostiene que una red residual de n bloques se comporta, desde el punto de vista del flujo de gradiente, como un **ensamble implícito** de 2^n caminos de distinta profundidad (cada bloque puede ser "atravesado" o "saltado" efectivamente, dependiendo de si $\mathcal{F}(x) \approx 0$). Esto explica por qué ResNet es notablemente robusto a la eliminación de bloques individuales durante la inferencia — una propiedad que las redes planas profundas no comparten en absoluto — y ofrece una explicación alternativa (además del argumento de optimización de la Sección 2) de por qué las conexiones residuales permiten profundidades extremas sin degradación.

7 Legado y Trayectoria de Diseño

	GoogLeNet (2014)	**ResNet-152 (2015)**	**Legado**
Innovación clave	Módulo Inception, reducción 1×1	Conexión residual	Ambas ideas se combinan en Inception-ResNet (2016)
Profundidad	22 capas	152 capas	ResNet permitió explorar profundidades de 1000+ capas experimentalmente
Normalización	Ninguna (BN aún no existía en Inception v1)	Batch Normalization	BN se volvió estándar universal tras ResNet
Clasificador	Global Average Pooling	Global Average Pooling	Adoptado permanentemente en arquitecturas posteriores

Las conexiones residuales se han convertido en un componente de diseño casi universal, mucho más allá de la clasificación de imágenes: son el mecanismo central detrás de arquitecturas Transformer (donde cada bloque de atención y cada bloque feed-forward incluye una conexión residual), redes generativas profundas, y esencialmente cualquier arquitectura moderna que supere las pocas decenas de capas.

8 Ejercicios

1. Derive la fórmula del número de parámetros de un bloque bottleneck en función de C_{in}, el factor de reducción r (típicamente $r = 4$), y C_{out}. Verifique su fórmula contra el Ejemplo de la Sección 2.3.

2. Explique por qué la conexión de atajo con relleno de ceros (Opción A) no agrega parámetros, mientras que la conexión de proyección (Opción B) sí lo hace. ¿Cuántos parámetros agrega una proyección 1×1 que cambia de $C_{\text{in}} = 128$ a $C_{\text{out}} = 256$ canales, con stride 2?

3. Usando la formulación $y = \mathcal{F}(x) + x$, explique matemáticamente por qué, si el gradiente de la pérdida respecto a y es $\frac{\partial \mathcal{L}}{\partial y}$, el gradiente respecto a x incluye un término que **no** depende de los pesos de $\mathcal{F}$. ¿Cómo previene esto el problema del gradiente que se desvanece en redes muy profundas?

4. Compare el número total de parámetros de ResNet-50 (25.6M) con VGG-16 (138M), a pesar de que ResNet-50 tiene más del triple de capas. Identifique los dos mecanismos de diseño (ya vistos en las notas de GoogLeNet y en éstas) que explican esta diferencia.

5. Investigue y explique brevemente por qué experimentos con redes residuales de más de 1000 capas (ResNet-1202 en el artículo original) mostraron un desempeño ligeramente *peor* que ResNet-110, a pesar de no sufrir del problema de degradación clásico. (Pista: piense en sobreajuste con un conjunto de entrenamiento de tamaño fijo, ahora que el problema de optimización ya no es el cuello de botella.)

9 Referencias

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Identity Mappings in Deep Residual Networks," *ECCV*, 2016.

[3] A. Veit, M. Wilber, and S. Belongie, "Residual Networks Behave Like Ensembles of Relatively Shallow Networks," *NeurIPS*, 2016.

[4] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning," *AAAI*, 2017.

[5] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," *ICML*, 2015.